DriftConditions Implementation Plan

Spring 2026 Operations Upgrade

Purpose

This implementation plan outlines the next wave of `MixEngine` improvements, backend infrastructure work, and stream quality fixes for DriftConditions. It is a companion to the Spring 2026 UI/UX Upgrade Implementation Plan and addresses work deferred from that effort — primarily items requiring `MixEngine` instrumentation, new database tables, or changes to how the station generates and manages its stream.

1. Goals and Scope

1.1. Goals

The primary goals of this upgrade are to:

1. **Improve Stream Quality and Musical Coherence:** Fix known bugs in clip selection, volume balance, duration calculation, and recipe behavior. Extend the filter system with missing capabilities.
2. **Instrument the `MixEngine` for Usage Tracking:** Record which clips are selected and played, enabling usage-weighted selection and surfacing play counts in the admin UI.
3. **Strengthen Backend Infrastructure:** Add audit logging for auth and user events, implement contributor notifications, and consolidate file-based logging.
4. **Explore Stream Expansion:** Investigate YouTube and Twitch output as a stretch goal.

1.2. Why these goals matter:

The `MixEngine` is the core of what DriftConditions does — everything the listener hears flows through it. Several known bugs degrade the quality of that experience: clips outside the requested length range are occasionally selected, volume balance between tracks is sometimes wrong, and certain recipes misbehave. Fixing these improves the station's output directly. Usage tracking closes a long-standing gap — the system currently has no memory of which clips it has played, making weighted selection and contributor feedback impossible. Infrastructure improvements reduce operational risk and lay groundwork for growth.

1.3. Scope

This upgrade focuses on server-side and `MixEngine` changes:

1.3.1. MixEngine Fixes

- Fix clip length selection — clips outside the requested range being chosen by `ClipSelector`
- Resolve intermittent NaN duration calculation
- Fix volume balance — background ambient tracks overpowering foreground elements
- Debug Long Narrative With Music Bed recipe misbehavior
- Fix normalization level on quiet clips

1.3.2. MixEngine Additions

- New filters: `fadeout`, `loop(n)`, `repeat(n)`, music bed ducking
- Recipe length weighting — prevent long environmental clips from dominating the stream
- Clip usage tracking — new `clipUsage` table; `MixEngine` instrumented to log each clip selected
- Usage-weighted clip selection — prefer less-used clips over time

1.3.3. Backend Infrastructure

- Audit table instrumentation — write auth and user change events (currently empty)
- Contributor digest email system — periodic summary of approvals and clip usage per user
- Logging consolidation — reduce or eliminate file-based logs in `log/*`

1.3.4. Stretch Goals

- YouTube / Twitch stream output

1.4. Out of Scope:

- Admin UI changes beyond what is directly enabled by new backend data (clip usage display on Audio View/Edit is specified in the UI/UX plan)

- Auth/authorization system changes (covered in the Auth Upgrade plan)
- New recipe authoring functionality

2. Implementation: MixEngine Fixes

The following tasks address known defects in the core clip selection, duration, and mixing logic. Each fix targets a specific failure mode that degrades the quality of the generated stream.

2.1. Clip Length Selection Correction

The `ClipSelector` component occasionally selects clips that fall outside the length range specified by the recipe — for example, choosing a 100-second clip when the recipe requests something between 2 and 10 minutes. The selection logic applies a range filter, but the filter is not always enforced strictly, likely due to a boundary condition or fallback path that bypasses the check.

The fix requires identifying the code path where out-of-range clips are returned and adding or correcting the length constraint. The expected result is that `ClipSelector` never returns a clip whose duration falls outside the recipe's specified `minLength`/`maxLength` bounds, with no silent fallback exceptions.

2.2. Resolve Intermittent NaN Duration Calculation

The `MixEngine` intermittently calculates an invalid (`NaN`) value for track duration, producing an `ffmpeg` error logged as `Unable to parse option value "NaN"`. This causes the affected mix to fail silently or produce a malformed output.

The root cause is likely in the expression evaluator or duration math when one or more input values is undefined or zero. The fix involves adding defensive validation at the point where durations are computed — ensuring that any `NaN`, `undefined`, or non-numeric result is caught before it propagates to the `ffmpeg` filter chain.

2.3. Volume Balance Correction

In some mixes, background ambient tracks overpower foreground elements — voices, narratives, or featured sounds — producing a sonically incoherent output. The current gain assignments in the filter chain do not reliably establish a hierarchy between background and foreground layers.

This task involves reviewing the relative gain values applied to background vs. foreground tracks and adjusting them to enforce a consistent sonic hierarchy. The fix should be applied at

the recipe or filter layer so that the relationship holds across different clip combinations without per-clip tuning.

Resolution: Close as won't fix at the code level. Recipe authors should ensure ambient/background tracks are set lower than foreground tracks. If automatic balancing becomes desirable in the future, it would require adding a `role` field to the track schema and corresponding logic in MixEngine to enforce relative volume constraints.

2.4. Debug Long Narrative With Music Bed Recipe

The "Long Narrative With Music Bed" recipe is not executing its intended logic correctly. The exact failure mode is not yet characterized — the recipe may be producing incorrect structure, wrong durations, or missing elements.

This task begins with diagnosis: running the recipe in isolation and comparing its actual output structure against its intended design. Once the root cause is identified, the fix will be scoped and applied. No implementation approach is specified here until diagnosis is complete.

Root cause identified and resolved. Historical `mixQueue` records showed two mixes with anomalous 6654s (110 min) durations. Both selected the same clip — "H1Z1 Survivor Radio Broadcast" at 6413s (106 min) — which exceeded the previous `huge` maximum of 3600s (60 min). The clip was reachable because the recipe at the time contained a malformed `length` key (`"long, huge"` as a single comma-separated string rather than `["long", "huge"]`), which caused the length filter to silently fail, allowing any-duration clip to be selected.

Resolution: Recipe corrected (separate strings). `huge` category max extended to 7200s (120 min) to accommodate legitimately long-form content. Warning logging added to `_constructLengthQuery` to surface future malformed keys.

2.5. Fix Normalization Level on Quiet Clips

Some audio clips — particularly quiet field recordings and spoken-word material — are normalized to a level too low for the mix, resulting in an inaudible or barely audible contribution to the stream. The normalization filter is applying a target that is insufficient for these clips.

The fix involves adjusting the normalization target to a louder standard and verifying that the result is audible in context without clipping or distortion. This may require a per-clip override path or a louder default target level.

Resolution: Investigation revealed that the normalization targets themselves were not the root cause. The `loudnorm` presets (`voice`: I=−16 LUFS, `bed`: I=−20 LUFS) were appropriate, but `ffmpeg`'s `amix` filter was silently undermining them. `amix` defaults to `normalize=1`, which scales each input by 1/N — with two tracks, every track's level is halved (~−6 dB) before the final

mix, directly undoing the per-clip normalization work. The fix detects whether any track or clip in the recipe uses a `norm` effect; if so, `amix` is set to `normalize=0`, allowing the pre-normalized levels to pass through unchanged. Recipes without norm effects retain the default behavior as a clipping safeguard. No changes to the `loudnorm` presets were required.

3. Implementation: MixEngine Additions

This section covers new capabilities being added to the `MixEngine` to increase stream variety, improve recipe expressiveness, and introduce the data infrastructure for usage tracking and weighted selection.

3.1. Audio Clip Usage Tracking

Recipe clips are selected dynamically at runtime by the `MixEngine` based on classification and length criteria. Individual audio clips are not referenced by ID in recipe data, so accurate usage tracking requires the `MixEngine` to instrument and log each clip it selects.

3.1.1. Infrastructure Changes

```sql
-- New table: tracks each clip actually heard in a mix
CREATE TABLE clipUsage (
  usageID  INT      NOT NULL AUTO_INCREMENT,
  audioID  INT      NOT NULL,
  recipeID INT      NOT NULL,
  usedAt   DATETIME DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (usageID),
  KEY idx_audioID  (audioID),
  KEY idx_recipeID (recipeID)
);

-- New column: cached play count on audio table for fast lookups
ALTER TABLE audio ADD COLUMN timesUsed INT DEFAULT 0;
```

A new `RecordKeeper` service is inserted into the `Conductor` pipeline immediately after `ClipAdjustor.adjustClipTimings()`, at which point all clip durations and `mixDuration`

are fully resolved. It walks each track accumulating elapsed time, identifies which clips fall within `mixDuration` (i.e., were actually heard), updates `audio.lastUsed` and increments `audio.timesUsed` for each heard clip, inserts a `clipUsage` row per heard clip, and returns an accurate playlist. Silence clips are excluded from logging; tiny clips are logged but excluded from the public-facing playlist.

This replaces `RecipeParser.getPlaylistFromRecipe()` and `ClipSelector._updateClipLastUsed()`, both of which operated on the full selected-clip list without accounting for mix truncation.

3.1.2. Considerations

- *clipUsage growth:* Flagged as a concern before launch. Resolved — `audio.timesUsed` serves as a counter-cache column, so `/api/audio/info` reads a single integer from `audio.*` rather than issuing a COUNT join against `clipUsage`. The detail table is retained for digest emails and analytics (3.4, 4.2).
- *Tracking at selection time vs. confirmed play time:* Selection is simpler to instrument; confirmed play requires stream feedback from Icecast. We implemented a third option — **heard time** — computed from `mixDuration` after timing adjustment. This is more accurate than selection time (clips cut off by a shorter mix are excluded) while avoiding the complexity of Icecast feedback.
- `clipUsage` also feeds usage-weighted clip selection (3.4) and the contributor digest email (4.2).

3.1.3. Clip Usage Count Display

- `timesUsed` returned from `/api/audio/info` directly from `audio.*` column
- Audio View and Audio Edit display of "Used X times" deferred to the UI/UX plan

3.2. Recipe Usage Tracking

Recipes vary significantly in the mixes they produce — some consistently generate short ambient pieces, others produce long-form compositions that can run an hour or more. Without tracking actual mix durations, the `RecipeSelector` has no way to account for this variation, and recipes that produce longer mixes occupy disproportionate airtime relative to how often they are selected.

3.2.1. Infrastructure Changes

```sql
-- New table: tracks each mix produced by a recipe
CREATE TABLE recipeUsage (
  usageID   INT      NOT NULL AUTO_INCREMENT,
  recipeID  INT      NOT NULL,
  duration  FLOAT    NOT NULL,
  usedAt    DATETIME DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (usageID),
  KEY idx_recipeID (recipeID)
);

-- New columns: cached play count and running average mix duration
ALTER TABLE recipes ADD COLUMN timesUsed   INT   DEFAULT 0;
ALTER TABLE recipes ADD COLUMN avgDuration FLOAT DEFAULT NULL;
```

The existing `RecordKeeper` service is extended to handle recipe tracking alongside clip tracking. After `mixDuration` is resolved, RecordKeeper:

- Increments `recipes.timesUsed` by 1
- Inserts a row into `recipeUsage` recording the `recipeID`, `duration`, and timestamp
- Updates `recipes.avgDuration` as a weighted running average:

```
avgDuration = (avgDuration × historyWeight + mixDuration) / (historyWeight + 1)
```

- If `avgDuration` is `NULL` (first run), `mixDuration` is stored directly rather than treating `NULL` as zero.

The history weight is stored in config (default: 10), meaning after roughly 10 runs the estimate is reasonably stable. Editing a recipe to use shorter or longer clips will gradually shift the average to reflect real-world behavior.

3.2.2. Considerations

- *Cold start:* `avgDuration` is `NULL` for all recipes until their first mix completes. `RecipeSelector` treats `NULL` as neutral — no duration penalty applied until data accumulates.

- *Variability:* `avgDuration` reflects actual mix durations, which vary run-to-run depending on clip selection. This is intentional — it captures real-world behavior rather than a theoretical estimate.

- `recipeUsage` and `avgDuration` feed recipe length weighting in `RecipeSelector` (3.5) and are available for future analytics.

3.2.3. Recipe Usage Display

- `timesUsed` and `avgDuration` available from the recipes API for display in Recipe View, Recipe Edit, and Recipe List (deferred to UI/UX plan).

3.3. Recipe Selection Weighted by Average Duration

Recipes vary considerably in the mixes they produce, and without accounting for that variation the `RecipeSelector` treats all candidate recipes as equally costly in terms of airtime. A recipe that consistently generates 60-minute mixes will occupy twice as much stream time as one that generates 30-minute mixes, even if both are selected at the same rate. Over a 24-hour period this compounds: the stream skews toward the output of a few long-running recipes rather than cycling broadly through the available library.

This task adds a duration-based subscore to the `RecipeSelector` that down-weights recipes whose `avgDuration` is high relative to the current candidate pool, giving shorter-producing recipes a proportionally higher chance of selection.

3.3.1. Infrastructure Changes

At selection time, `RecipeSelector` queries `MIN` and `MAX` `avgDuration` across the eligible recipe pool. Each candidate recipe receives a duration subscore:

```
durationScore = (maxDuration - avgDuration) / (maxDuration -
minDuration)
```

This yields a value in [0, 1] where recipes near the minimum average duration score 1.0 and recipes near the maximum score 0.0. The score is combined with existing subscores (recency, classification diversity) as a weighted sum. The weight scalar is stored in config under `recipes.durationScoreWeight`.

Recipes with `NULL avgDuration` — those that have not yet generated a mix — receive a neutral score (0.5) so they are neither favored nor penalized before data accumulates.

3.3.2. Considerations

- **One factor among many:** A long-average recipe is nudged toward lower selection probability, not disqualified — it will still be chosen when other factors favor it.
- **Cold start:** `NULL avgDuration` scores 0.5 (true neutral). Using 1.0 would unfairly favor unproven recipes over established shorter ones.
- **Degenerate pool:** If all eligible recipes share the same `avgDuration`, score defaults to 0.5 to avoid division by zero.
- **Depends on `avgDuration` data from 3.2;** will self-calibrate as more runs accumulate.

3.4. Clip Selection Weighted by Times Used

The clip pool for any given recipe may contain many qualifying clips, but without usage-awareness the `ClipSelector` draws from it without regard to how often individual clips have already aired. Popular or well-tagged clips can dominate the rotation while lesser-used clips rarely surface.

This task adds a usage-based weight to clip selection so that clips with lower `timesUsed` counts are proportionally more likely to be chosen, broadening the effective rotation without excluding any clip entirely.

3.4.1. Infrastructure Changes

At clip selection time, ClipSelector computes MIN and MAX `timesUsed` across the eligible clip pool. Each candidate clip receives a usage subscore:

```
usageScore = (maxUsed - timesUsed) / (maxUsed - minUsed)
```

Clips with `timesUsed = 0` or NULL score 1.0 (never played = highest priority). The score is combined with the existing subscores (recency, tag match) as a weighted sum. The weight scalar is stored in config under `audio.usageScoreWeight`.

Edge cases:

- **Never-used clips:** Score 1.0 — new uploads get natural exposure before any data accumulates.
- **Degenerate pool:** If all clips have the same `timesUsed`, score defaults to 0.5 to avoid division by zero.
- **Depends on `timesUsed` data from 3.1;** will self-calibrate as more runs accumulate.

3.4.2. Considerations

- **Depends on `timesUsed` data from 3.1;** will self-calibrate as more runs accumulate.
- **Degenerate pool:** If all eligible clips have the same `timesUsed`, score defaults to neutral to avoid division by zero.
- **Not a hard exclusion:** Heavily-used clips remain eligible — they are just less likely to be chosen when fresher alternatives exist.

3.5. New Audio Filters

We are adding four new filters to extend the expressive range of recipes:

- **`fadeout`** — fades a clip's audio out at its end rather than cutting abruptly. Useful for ambient beds and narrative closings.
- **`loop(n)`** — repeats a clip `n` times end-to-end, producing a longer continuous segment from a short source clip.
- **`repeat(n)`** — reuses a clip that was already selected earlier in the mix, avoiding a second random selection. Useful for call-and-response or motif structures.
- **Music bed ducking** — reduces the level of a background music bed automatically to follow the dynamic envelope of a foreground track. Replaces the current static gain approach for layered compositions.

Each filter is added to the `MixEngine`'s filter registry and must be testable independently before being used in production recipes.

4. Implementation: Backend Infrastructure

These tasks strengthen the core operational foundations of the station, reduce long-term operational risk, and improve the contributor experience through automated communication.

4.1. Audit Table Instrumentation

An `audit` table exists in the database but is currently empty — no code writes to it. This task instruments the auth and user management routes to write records for key security and administrative events.

Events to log: successful and failed sign-ins, sign-outs, user role changes, user status changes, password resets, and OAuth account linking. Each record should include the acting user, the affected user (if different), the event type, a timestamp, and any relevant metadata.

This provides a basic operational audit trail for debugging, moderation review, and security incident response.

4.2. Contributor Digest Email System

Rather than individual per-approval emails, a periodic digest re-engages contributors with a summary of their recent activity on the station. A contributor who submits multiple clips in one session receives one digest, not multiple emails.

Digest content (personalized per user):

- Username and link to profile
- Current role
- Member since date
- Since last digest: clips approved, clips used in recipes (requires 3.3)

Implementation:

- `userComms` table accumulates communication-worthy events (audio approved, clip used, role changed) with a `sentAt` field gating against re-sending
- Digest builder groups pending `userComms` by user and renders a personalized email
- Scheduler runs the digest on a configurable interval (weekly default); only sends if there is at least one pending event
- Uses existing `sendMail` from `utils/mailer.js`
- Users can opt out via a `digestEmail` boolean on the `users` table (default `true`)

Dependencies: The "clips used in recipes" digest row requires 3.3 (clip usage tracking). The digest can launch without it and add that section once tracking is live.

4.3. Logging Consolidation

The system currently writes logs to multiple files in `log/`. This creates operational overhead — log rotation, disk management, and debugging require checking multiple files. The goal is to reduce or eliminate file-based logging in favor of a more manageable approach (stdout/stderr captured by the process manager, or a single consolidated log).

This task involves auditing what is currently written to each file in `log/`, determining what is still needed, and migrating or eliminating the rest. No log data that is currently useful for operations or debugging should be lost without a replacement.

Resolution: Already complete. The file transport in `config/logger.js` was commented out in a prior session, leaving console-only output. Since `adminserver` and `mixengine` run as systemd services, all console output is captured by journald automatically. The `logs/` directory still exists with a few old log files from before the file transport was disabled, but no new files are being written. No code changes required; this item is closed.

4.4. Rename Legacy Infrastructure

The project originated under the name `interference` — a working title that predates the public identity of `DriftConditions`. While the station's public-facing name has been established for some time, the legacy name persists in several places across the development and production infrastructure. This task replaces those references systematically.

Locations requiring rename:

- Production database — named `interference` on the server
- Local development database — named `interference` locally
- `config/config.js` — database name hardcoded as `interference`
- Root `package.json` / `package-lock.json` — `name`: `interference`
- `.env` (root, local) — `BASEDIR` points to `~/dev/interference/`
- `setupfiles/env.server` — `BASEDIR` and inline comment paths reference `~/interference`
- `setupfiles/env.local` — same
- `scripts/rebuild-client.sh` — directory name check hardcoded to `interference`
- `scripts/user-activity-30-days.sh` — `DB_NAME="interference"` and ssh path
- `AdminClient/src/App.js` — user-visible heading: `<h1>interference</h1>`
- `README.md` — project name and prose references
- Local repo directory — `~/dev/interference/`
- Production server repo directory — `~/interference/`

The new name throughout will be `driftconditions`.

4.4.1. Rename Approach

Code and config changes are straightforward and low-risk. The database rename and directory renames require more care and should be done in a deliberate order. The recommended sequence is:

1. Update all code and config references (`config.js`, `package.json`, scripts, env files, `App.js`, `README.md`)
2. Rename the local database (`interference` → `driftconditions`) and verify locally
3. Deploy code changes to the server
4. Rename the production database on the server
5. Rename the production repo directory (`~/interference` → `~/driftconditions`) and update any symlinks or service files that reference it
6. Rename the local repo directory last (or leave it — it has no runtime effect)

4.4.2. Interruption Mitigation

The production database rename and directory rename are the two steps most likely to cause a service interruption. Both should be treated as a brief, planned downtime.

Database rename: `MySQL` does not support a single `RENAME DATABASE` command. The safe approach is to create a new database, dump and restore the data, verify, then drop the old name. The `MixEngine` and `AdminServer` must both be stopped during the restore to prevent writes to the old database. Total downtime should be under five minutes.

Directory rename on the server: All service files, reverse proxy configs, and environment configs that reference `~/interference` as a path are hosted in `setupfiles/` and symlinked into place on the server. This means every path reference can be updated here in the source before deployment. The complete set of files requiring updates is: `env.server`, `adminserver.service`, `mixengine.service`, `liquidsoap.service`, `Caddyfile.server`, and `icecast-server.xml`. Once those are updated and deployed, the service is stopped, the directory renamed, and the service restarted in a single operation.

Rollback: Until the old database is dropped and the old directory is removed, rollback is a matter of reverting the config file changes and restarting services. Drop and cleanup should not happen until everything has been verified running under the new name.

5. Stretch Goals

5.1. Thumbs Up/Down

This feature implements a listener rating mechanism within the playlist component. A listener can give a thumbs up or thumbs down to the currently playing mix. The response is recorded and associated with that playlist entry.

A thumbs-up rating also flags the playlist so it is not pruned by the prune mixes job, preserving well-received mixes in the archive.

5.2. User List Quick Edit

Status: Complete. An inline quick-edit row was added to the User List page in a prior commit. Role and Status dropdowns are available without leaving the page.

5.3. YouTube / Twitch Stream Output

This stretch goal investigates the technical feasibility of routing the generated stream to YouTube and/or Twitch in addition to the existing Icecast output. The deliverable is a feasibility assessment — what is required, what the constraints are, and whether it is worth pursuing — not necessarily a working implementation.

6. Risk Management and Fallback Plan

7. Verification and Success Criteria

8. Appendices

8.1. Appendix A — Database Schema Diff

8.2. Appendix B — Server Changes

8.3. Appendix C — Future Enhancements and Technical Debt

8.4. Appendix D — Document Style Guide

This section details the standardized formatting applied to the document.

8.4.1. Headers and Section Titles

This style dictates the formatting for all section titles and headings (Heading 1, Heading 2, Heading 3, etc.) to ensure clear hierarchy and visual distinction from body text. All headers must be formatted exclusively using these standard Google Docs heading styles rather than custom text styles.

- **Font Family:** Arial
- **Font Size:** Varied (e.g., 20pt for Heading 1, 16pt for Heading 2, 14pt for Heading 3)
- **Font Weight:** Bold
- **Application:** Applied exclusively to section titles and subtitles, using the designated Docs Heading styles.
- **Structural Patterns:** H1 headings (e.g., "1. Goals and Scope") must use the "Heading 1" doc style; H2 headings (e.g., "1.1. Goals") must use the "Heading 2" doc style; and this hierarchical pattern must be followed for all subsequent heading levels.

8.4.2. Body Paragraphs and Standard Prose

This section defines the baseline formatting for all narrative and descriptive text in the document. Standardized prose ensures optimal readability and consistency across all sections, distinguishing it clearly from headings and technical lists.

- **Font Family:** Arial
- **Font Size:** 11pt
- **Line Spacing:** 1.15
- **Spacing (Before Paragraph):** 0pt
- **Spacing (After Paragraph):** 12pt
- **Application:** Applied exclusively to normal body text, excluding headings, titles, and list items.

8.4.3. Inline Technical References

This style is used for all short code references embedded within the main text, such as variable names, file paths, API routes, and terminal commands. It serves to visually isolate code elements from surrounding narrative text.

- **Font Family:** Roboto Mono (Monospaced)
- **Text Color:** Gemini Green (#188038)
- **Background:** None
- **Use Cases:** Inline code snippets (e.g., `clipUsage` table), variable names (e.g., `audioID`), file paths (e.g., `config/config.js`), and console output text.

8.4.4. Code Block Syntax Highlighting Style: One Light (Notion variant)

A light-background syntax theme derived from the One Light family, originally developed for the

Atom editor. Clean, low-contrast, and readable on white or near-white backgrounds.

Background: Light gray (`#f7f7f7` or similar off-white) **Default text:** Dark gray (identifiers, punctuation, table/column names)

Token colors:

- **Keywords** (`ALTER`, `CREATE`, `ADD`, `NOT`, `NULL`, `ON`, `DELETE`): Medium blue (`#0070c1` or similar)
- **Types** (`INT`, `VARCHAR`, `DATETIME`): Bright blue (`#0099cc` or similar)
- **Numeric literals** (`255`, `32`, `191`): Orange-red (`#e06c00` or similar)
- **Comments** (`-- ...`): Muted pink/mauve (`#c678a0` or similar)
- **Language label** (`SQL`): Small-caps, light gray, non-highlighted

Typography: Monospace, medium weight, generous line height. Token colors are distinct but not garish — optimized for readability over visual drama.

```sql
-- Existing 'users' table update (only adding utility columns)
ALTER TABLE users
  ADD COLUMN displayName VARCHAR(255),
  ADD COLUMN lastLoginAt DATETIME;

-- New 'userIdentities' table to support multiple linked OAuth accounts
CREATE TABLE userIdentities (
  userId INT NOT NULL, -- Foreign key to the users table
  oauthProvider VARCHAR(32) NOT NULL,
  oauthId VARCHAR(191) NOT NULL,
  PRIMARY KEY (oauthProvider, oauthId),
  FOREIGN KEY (userId) REFERENCES users(userID) ON DELETE CASCADE
);

-- Allow null names since OAuth providers may not supply them
ALTER TABLE users
```